

آزمایشگاه طراحی سیستم های دیجیتال

دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف



دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف

بهار 1405

استاد: دکتر انصاری

اعضای گروه:

کیمیا قدیر 402106326

کیمیا رضائی 402106048

ملینا طاهری 402106167

موضوع :

آزمایش سوم-توصیف جریان داده

هدف آزمایش:

در این آزمایش هدف آشنایی با طراحی مدارات ترکیبی و ترتیبی به روش Dataflow Modeling در Verilog است. در این راستا دو مدار مقایسه کننده طراحی و بررسی می شوند:

- مقایسه کننده قابل گسترش 1 بیتی و ساخت نمونه 4 بیتی از روی آن
- مقایسه کننده سریال همچنین صحت عملکرد مدارها با استفاده از تست بنچ های خودکار بررسی می شود.

شرح کلی آزمایش:

در این آزمایش از دستور assign برای توصیف رفتار مدارها استفاده شده است. در بخش اول، یک مقایسه کننده 1 بیتی طراحی می شود که سه خروجی بزرگ تر بودن، مساوی بودن و کوچک تر بودن را تولید می کند و علاوه بر آن دارای ورودی های کمکی برای اتصال آپشاری به طبقات دیگر است. سپس با کنار هم قرار دادن چهار نمونه از این مدار، یک مقایسه کننده 4 بیتی ساخته می شود. در بخش دوم نیز یک مقایسه کننده سریال طراحی می شود که با دریافت بیت ها به صورت ترتیبی و با استفاده از کلاک و ریست، نتیجه مقایسه را در طول زمان تعیین می کند.

مراحل انجام آزمایش:

مرحله اول:

در این بخش یک مقایسه‌کننده 1 بیتی طراحی می‌شود که علاوه بر دو ورودی اصلی a و b ، سه ورودی کمکی با نام‌های gr_in ، eq_in و ls_in دارد. این ورودی‌ها نتیجه مقایسه بیت‌های با ارزش بالاتر را از مرحله قبلی دریافت می‌کنند. منطق مدار به این صورت است که اگر بیت a از بیت b بزرگ‌تر باشد و تا مرحله قبل نیز دو عدد برابر فرض شده باشند، خروجی gr فعال می‌شود. اگر بیت a از بیت b کوچک‌تر باشد و تا قبل از آن برابری برقرار بوده باشد، خروجی ls فعال می‌شود. در حالتی که دو بیت برابر باشند، نتیجه قبلی بدون تغییر به مرحله بعد منتقل می‌شود. بنابراین این مدار می‌تواند به صورت آبشاری در کنار چند مقایسه‌کننده دیگر قرار گیرد و مقایسه اعداد چند بیتی را انجام دهد. استفاده از `assign` باعث شده کل رفتار مدار به صورت مستقیم و مبتنی بر روابط منطقی توصیف شود.

```
1 module cascaded_comparator(input in_gr, input in_eq, input in_le, input a, input b, output out_gr, output out_eq, output out_le);  
2     assign out_gr = in_gr | (in_eq & (a & (~b)));  
3     assign out_eq = in_eq & ~(a ^ b);  
4     assign out_le = in_le | (in_eq & ((~a) & b));  
5 endmodule
```

مرحله دوم:

پس از طراحی مقایسه‌کننده 1 بیتی ، برای اطمینان از صحت عملکرد مدار یک تست‌بنچ خودکار (Self-Checking Testbench) طراحی شد. در این تست‌بنچ تمامی ترکیب‌های مهم ورودی شامل سیگنال‌های کمکی `in_eq`، `in_gr` و `in_le` به همراه ورودی‌های اصلی `a` و `b` بررسی می‌شوند.

در این تست‌بنچ ابتدا مقادیر ورودی به مدار اعمال می‌شوند و پس از یک تأخیر زمانی کوتاه، خروجی‌های مدار با مقادیر مورد انتظار مقایسه می‌شوند. مقادیر مورد انتظار بر اساس روابط منطقی مدار محاسبه شده‌اند:

- اگر تا مرحله قبل نتیجه بزرگ‌تر بودن مشخص شده باشد، خروجی `out_gr` فعال باقی می‌ماند.

- اگر تا مرحله قبل برابری برقرار باشد و $b < a$ شود، خروجی `out_gr` فعال می‌شود.

- اگر دو بیت برابر باشند، خروجی `out_eq` فعال باقی می‌ماند.

- اگر تا مرحله قبل برابری برقرار باشد و $b > a$ شود، خروجی `out_le` فعال می‌شود.

در این تست‌بنچ از یک `task` با نام `run_test` استفاده شده است تا اجرای تست‌ها به صورت منظم و قابل تکرار انجام شود. برای هر تست، خروجی واقعی مدار با خروجی مرجع مقایسه می‌شود و نتیجه به صورت `PASS` یا `FAIL` نمایش داده می‌شود.

```
Transcript
add wave -position insertpoint sim:/tb_castcadable_comparator_auto/*
VSIM 14> run -all
# PASS | in_gr=0 in_eq=1 in_le=0 a=0 b=0 || out=010
# PASS | in_gr=0 in_eq=1 in_le=0 a=0 b=1 || out=001
# PASS | in_gr=0 in_eq=1 in_le=0 a=1 b=0 || out=100
# PASS | in_gr=0 in_eq=1 in_le=0 a=1 b=1 || out=010
# PASS | in_gr=1 in_eq=0 in_le=0 a=0 b=0 || out=100
# PASS | in_gr=1 in_eq=0 in_le=0 a=0 b=1 || out=100
# PASS | in_gr=1 in_eq=0 in_le=0 a=1 b=0 || out=100
# PASS | in_gr=1 in_eq=0 in_le=0 a=1 b=1 || out=100
# PASS | in_gr=0 in_eq=0 in_le=1 a=0 b=0 || out=001
# PASS | in_gr=0 in_eq=0 in_le=1 a=0 b=1 || out=001
# PASS | in_gr=0 in_eq=0 in_le=1 a=1 b=0 || out=001
# PASS | in_gr=0 in_eq=0 in_le=1 a=1 b=1 || out=001
# PASS | in_gr=0 in_eq=0 in_le=0 a=0 b=0 || out=000
# PASS | in_gr=0 in_eq=0 in_le=0 a=0 b=1 || out=000
# PASS | in_gr=0 in_eq=0 in_le=0 a=1 b=0 || out=000
# PASS | in_gr=0 in_eq=0 in_le=0 a=1 b=1 || out=000
# PASS | in_gr=1 in_eq=1 in_le=1 a=0 b=0 || out=111
```

```
# PASS | in_gr=0 in_eq=0 in_le=1 a=1 b=0 || out=001
# PASS | in_gr=0 in_eq=0 in_le=1 a=1 b=1 || out=001
# PASS | in_gr=0 in_eq=0 in_le=0 a=0 b=0 || out=000
# PASS | in_gr=0 in_eq=0 in_le=0 a=0 b=1 || out=000
# PASS | in_gr=0 in_eq=0 in_le=0 a=1 b=0 || out=000
# PASS | in_gr=0 in_eq=0 in_le=0 a=1 b=1 || out=000
# PASS | in_gr=1 in_eq=1 in_le=1 a=0 b=0 || out=111
#
=====
# Total Tests   = 17
# Passed Tests  = 17
# Failed Tests  = 0
#
=====
# ** Note: Settop   : C:/Users/Lenovo/Desktop/dad3/tb_castcadable_comparator_auto.v(92)
# Time: 17 ns Iteration: 0 Instance: /tb_castcadable_comparator_auto
# Break in Module tb_castcadable_comparator_auto at C:/Users/Lenovo/Desktop/dad3/tb_castcadable_comparator_auto.v line 92
```

مرحله سوم:

پس از طراحی بلوک 1 بیتی، چهار نمونه از آن برای ساخت مقایسه‌کننده 4 بیتی به هم متصل می‌شوند. مقایسه از بیت پرارزش‌تر آغاز می‌شود، زیرا در اعداد دودویی بیت‌های سمت چپ اهمیت بیشتری دارند. در ابتدا به مرحله اول این‌گونه القا می‌شود که دو عدد تاکنون برابر بوده‌اند، بنابراین `eq_in` برابر 1 و دو ورودی دیگر صفر در نظر گرفته می‌شوند. خروجی هر مرحله به عنوان ورودی کمکی مرحله بعدی مورد استفاده قرار می‌گیرد. اگر در یکی از بیت‌های بالاتر مشخص شود که یکی از دو عدد بزرگ‌تر است، این نتیجه به مراحل بعدی منتقل می‌شود و بیت‌های کم‌ارزش‌تر دیگر نمی‌توانند نتیجه را تغییر دهند. این ساختار دقیقاً همان مفهوم Cascadable بودن مدار را نشان می‌دهد.

```
1 module comparator (input [3:0] a, input [3:0] b, output greater, output equal, output less);
2
3     wire [4:0] gr;
4     wire [4:0] eq;
5     wire [4:0] le;
6
7     assign gr[4] = 1'b0;
8     assign eq[4] = 1'b1;
9     assign le[4] = 1'b0;
10
11     genvar i;
12     generate
13         for(i = 4; i > 0; i = i - 1)begin: cast
14             castcadable_comparator _cast(
15                 .in_gr(gr[i]),
16                 .in_eq(eq[i]),
17                 .in_le(le[i]),
18                 .a(a[i - 1]),
19                 .b(b[i - 1]),
20                 .out_gr(gr[i - 1]),
21                 .out_eq(eq[i - 1]),
22                 .out_le(le[i - 1])
23             );
24         end
25     endgenerate
26     assign greater = gr[0];
27     assign equal = eq[0];
28     assign less = le[0];
29 endmodule
```

مرحله چهارم:

در فایل تست خودکار مربوط به مقایسه‌کننده 4 بیتی، تمام حالت‌های ممکن برای دو ورودی بررسی می‌شوند. از آنجا که هر ورودی 4 بیت است، هر کدام می‌توانند 16 مقدار مختلف داشته باشند؛ بنابراین در مجموع 256 ترکیب ورودی مختلف وجود دارد که باید آزمایش شوند.

منطق عملکرد تست‌بنچ به این صورت است:

ابتدا دو متغیر شمرانده برای تولید مقادیر ورودی تعریف می‌شوند. سپس با استفاده از دو حلقه تو در تو تمام مقادیر ممکن برای دو ورودی تولید می‌شوند. در حلقه اول مقدار متغیر از 0 تا 15 تغییر می‌کند و در حلقه دوم مقدار متغیر زنیز از 0 تا 15 تغییر داده می‌شود. در هر تکرار از حلقه‌ها، مقدار این متغیرها به صورت دودویی به ورودی‌های a و b اعمال می‌شود.

پس از اعمال هر ترکیب ورودی، یک تأخیر زمانی کوتاه در نظر گرفته می‌شود تا خروجی مدار پایدار شود. سپس خروجی‌های مدار یعنی greater، equal و less با مقادیر مرجع مقایسه می‌شوند.

مقادیر مرجع با استفاده از عملگرهای مقایسه‌ای خود زبان Verilog محاسبه می‌شوند. به این صورت که اگر مقدار عددی بزرگ‌تر از زباشد خروجی greater باید فعال شود، اگر دو مقدار برابر باشند خروجی equal باید فعال باشد و اگر کوچک‌تر از زباشد خروجی less باید فعال شود.

در صورتی که خروجی مدار با مقدار مورد انتظار برابر باشد، آن آزمایش به عنوان PASS ثبت می‌شود و در غیر این صورت FAIL گزارش می‌شود. در پایان شبیه‌سازی نیز تعداد کل تست‌ها، تعداد تست‌های موفق و تعداد خطاها نمایش داده می‌شود.

با توجه به اینکه در این تست‌بنچ تمام 256 حالت ممکن بررسی می‌شوند، می‌توان از صحت عملکرد مقایسه‌کننده 4 بیتی طراحی شده اطمینان حاصل کرد.

```
Transcript
VSIM 2> run -all
# PASS | a=0000 b=0000 || out=010
# PASS | a=0000 b=0001 || out=001
# PASS | a=0000 b=0010 || out=001
# PASS | a=0000 b=0011 || out=001
# PASS | a=0000 b=0100 || out=001
# PASS | a=0000 b=0101 || out=001
# PASS | a=0000 b=0110 || out=001
# PASS | a=0000 b=0111 || out=001
# PASS | a=0000 b=1000 || out=001
# PASS | a=0000 b=1001 || out=001
# PASS | a=0000 b=1010 || out=001
# PASS | a=0000 b=1011 || out=001
# PASS | a=0000 b=1100 || out=001
# PASS | a=0000 b=1101 || out=001
# PASS | a=0000 b=1110 || out=001
# PASS | a=0000 b=1111 || out=001
```

```
# PASS | a=0000 b=1110 || out=001
# PASS | a=0000 b=1111 || out=001
# PASS | a=0001 b=0000 || out=100
# PASS | a=0001 b=0001 || out=010
# PASS | a=0001 b=0010 || out=001
# PASS | a=0001 b=0011 || out=001
# PASS | a=0001 b=0100 || out=001
# PASS | a=0001 b=0101 || out=001
# PASS | a=0001 b=0110 || out=001
# PASS | a=0001 b=0111 || out=001
# PASS | a=0001 b=1000 || out=001
# PASS | a=0001 b=1001 || out=001
# PASS | a=0001 b=1010 || out=001
# PASS | a=0001 b=1011 || out=001
# PASS | a=0001 b=1100 || out=001
# PASS | a=0001 b=1101 || out=001
# PASS | a=0001 b=1110 || out=001
```

```
Transcript
# PASS | a=1111 b=1000 || out=100
# PASS | a=1111 b=1001 || out=100
# PASS | a=1111 b=1010 || out=100
# PASS | a=1111 b=1011 || out=100
# PASS | a=1111 b=1100 || out=100
# PASS | a=1111 b=1101 || out=100
# PASS | a=1111 b=1110 || out=100
# PASS | a=1111 b=1111 || out=010
# =====
# Total Tests   = 256
# Passed Tests  = 256
# Failed Tests  = 0
# =====
# ** Note: $stop      : C:/Users/Lenovo/Desktop/dsd3/tb_comparator_auto.v(64)
#    Time: 256 ns  Iteration: 0  Instance: /tb_comparator_auto
# Break in Module tb_comparator_auto at C:/Users/Lenovo/Desktop/dsd3/tb_comparator_auto.v line 64
```

اینجا چون خروجی خیلی زیاد بود از اول و آخرش اسکرین شات گذاشتیم که شمرده هم میشه توی عکس مشخصه هر 256 تا تست پاس شدن

مرحله پنجم:

در این بخش یک مقایسه‌کننده سریال طراحی می‌شود که بیت‌های دو عدد را به صورت ترتیبی و در چند سیکل کلاک دریافت می‌کند. این مدار دارای ورودی‌های `clk` و `reset` و همچنین دو ورودی تک‌بیتی `a` و `b` است. سه خروجی `eq`، `gr` و `less` وضعیت مقایسه را نشان می‌دهند.

منطق اصلی این مدار بر پایه نگهداری نتیجه مقایسه تا لحظه فعلی است. در ابتدای کار و بعد از فعال شدن `reset` فرض می‌شود دو عدد برابر هستند، بنابراین `eq` برابر 1 و دو خروجی دیگر صفر می‌شوند. سپس در هر لبه کلاک، یک بیت از هر ورودی دریافت می‌شود. اگر تا قبل از این لحظه دو عدد برابر بوده‌اند و در این مرحله `a` بزرگ‌تر از `b` باشد، خروجی `gr` فعال خواهد شد. اگر `a` کوچک‌تر از `b` باشد، خروجی `less` فعال می‌شود. اما اگر در مراحل قبلی مشخص شده باشد که یکی از دو عدد بزرگ‌تر است، این نتیجه در سیکل‌های بعدی حفظ می‌شود و بیت‌های بعدی دیگر تأثیری بر آن ندارند. به این ترتیب مدار به صورت ترتیبی و با حافظه داخلی، مقایسه دو عدد را به صورت سریال انجام می‌دهد.

```
1  module serial_comparator (  
2      input  wire a,  
3      input  wire b,  
4      input  wire clk,  
5      input  wire rst,  
6      output wire greater,  
7      output wire equal,  
8      output wire less  
9  );  
10  
11      wire g_b;  
12      wire e_b;  
13      wire l_b;  
14  
15      assign g_b = greater | (equal & a & ~b);  
16      assign e_b = equal & ~(a ^ b);  
17      assign l_b = less | (equal & ~a & b);  
18  
19      assign greater = (rst) ? 1'b0 : ((clk) ? g_b : greater);  
20      assign equal   = (rst) ? 1'b1 : ((clk) ? e_b : equal);  
21      assign less    = (rst) ? 1'b0 : ((clk) ? l_b : less);  
22  
23  endmodule  
24
```

مرحله ششم:

تست‌بنچ مربوط به مقایسه‌کننده سریال برای بررسی عملکرد مدار در شرایط زمانی مختلف طراحی شده است. در ابتدای این تست‌بنچ یک سیگنال clock تولید می‌شود که به طور پیوسته تغییر وضعیت می‌دهد و باعث می‌شود مدار در هر پالس ساعت بیت جدیدی را دریافت کند.

در ادامه برای هر جفت عدد 4 بیتی، ابتدا سیگنال reset برای مدت کوتاهی فعال می‌شود تا مدار به حالت اولیه بزرگردد. در این حالت اولیه فرض می‌شود که دو عدد با هم برابر هستند، بنابراین خروجی equal فعال و دو خروجی دیگر غیرفعال هستند.

پس از غیر فعال شدن reset، بیت‌های دو عدد ورودی به صورت سریال و از بیت پرازش‌تر به بیت کم‌ارزش‌تر به مدار اعمال می‌شوند. در هر چرخه clock یک بیت از هر عدد روی ورودی‌های a و b قرار می‌گیرد. این کار معمولاً در لبه نزولی کلاک انجام می‌شود تا در لبه بعدی کلاک مدار بتواند مقدار جدید را پردازش کند.

در طول این فرآیند، مدار با توجه به بیت‌های دریافت شده و حالت قبلی خود، وضعیت مقایسه را به‌روزرسانی می‌کند. به این ترتیب خروجی‌ها نشان می‌دهند که تا آن لحظه کدام عدد بزرگ‌تر، کوچک‌تر یا برابر است.

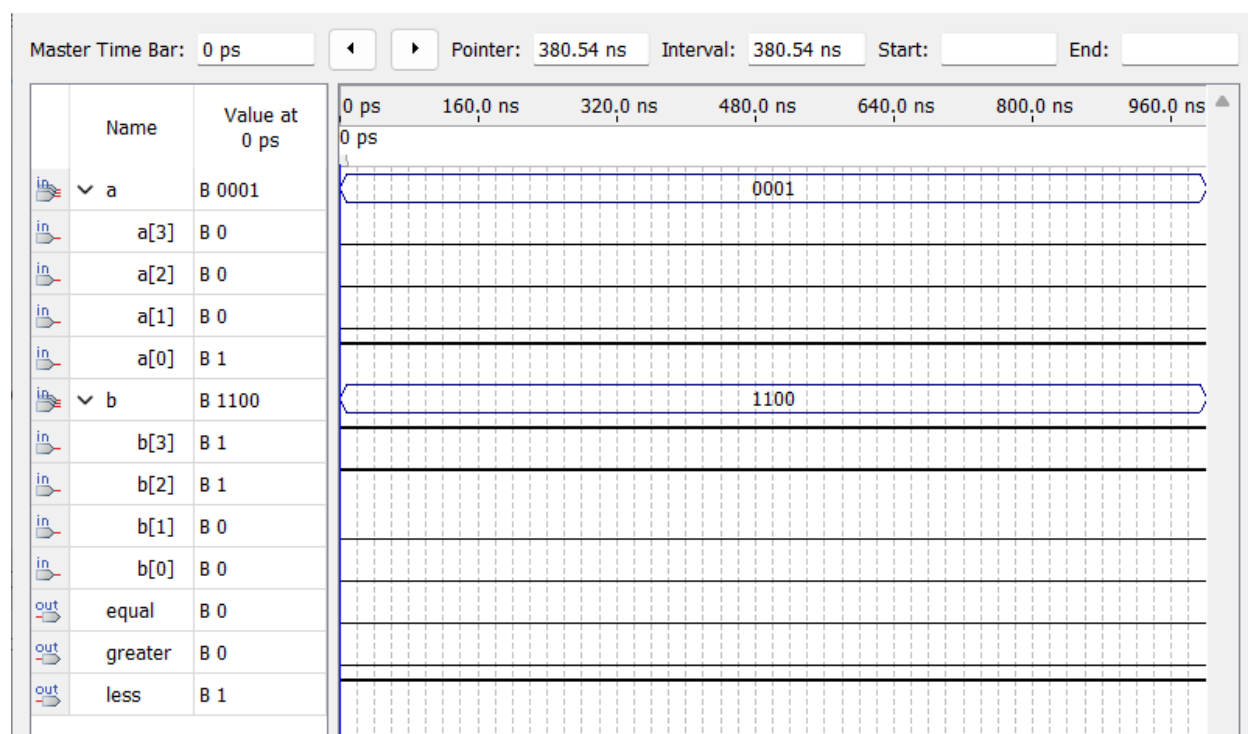
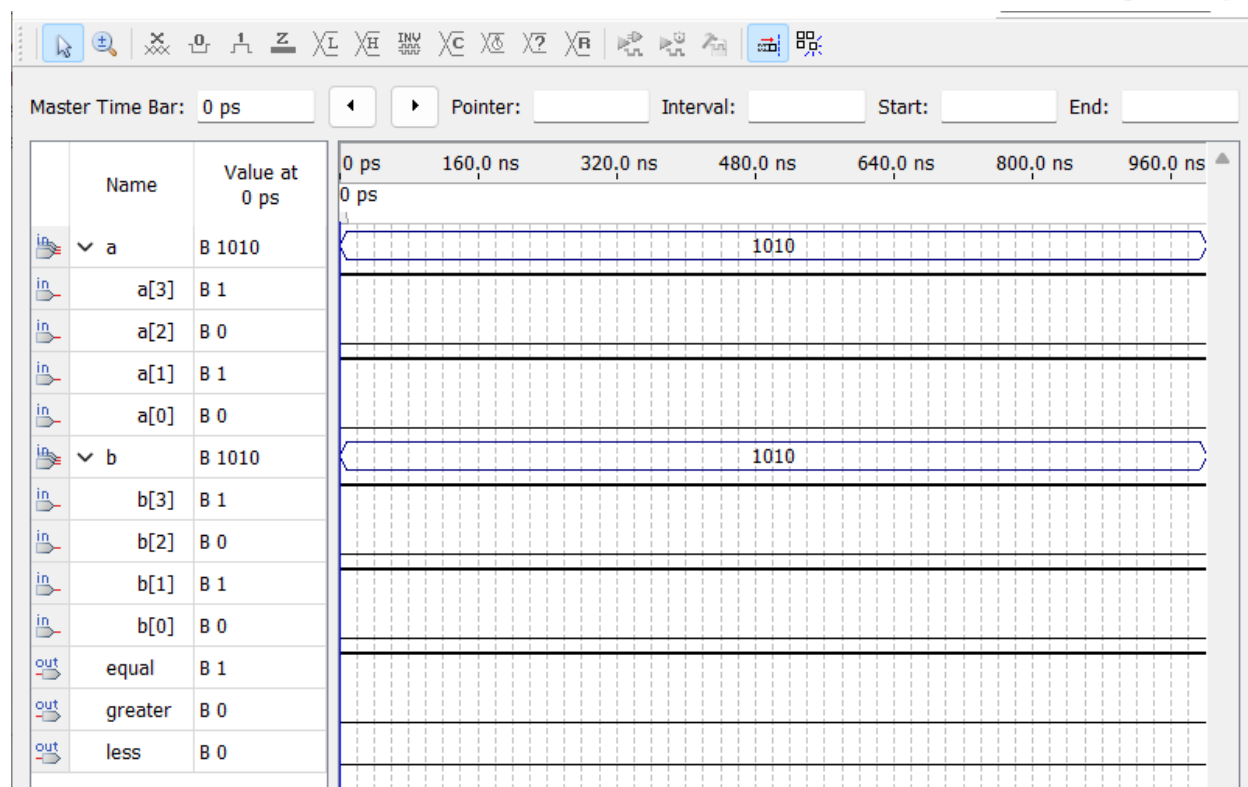
پس از ارسال تمام بیت‌ها، نتیجه نهایی مقایسه با مقدار مرجع که با استفاده از عملگرهای مقایسه‌ای Verilog محاسبه شده است، مقایسه می‌شود. در صورتی که خروجی مدار با نتیجه مورد انتظار یکسان باشد، تست موفق در نظر گرفته می‌شود و در غیر این صورت خطا گزارش می‌شود.

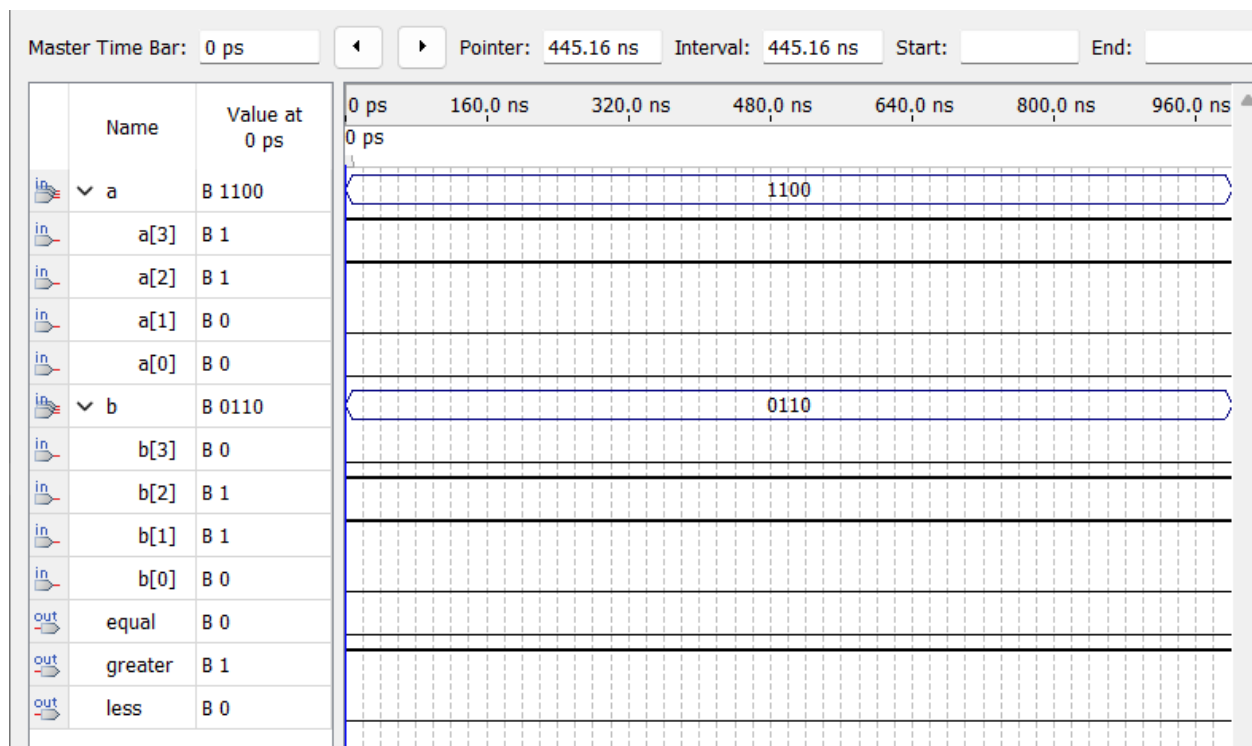
در این تست‌بنچ چندین بردار ورودی مختلف بررسی می‌شوند تا عملکرد مدار سریال در حالت‌های مختلف مقایسه مورد ارزیابی قرار گیرد.

```
Transcript
VSIM 5> run -all
# PASS | A=0000 B=0000 || out=010
# PASS | A=0001 B=0000 || out=100
# PASS | A=0000 B=0001 || out=001
# PASS | A=1011 B=1001 || out=100
# PASS | A=0110 B=0110 || out=010
# PASS | A=1111 B=0111 || out=100
# PASS | A=0011 B=1010 || out=001
# PASS | A=1000 B=1001 || out=001
# =====
# Total Tests   = 8
# Passed Tests  = 8
# Failed Tests  = 0
# =====
# ** Note: $stop      : C:/Users/Lenovo/Desktop/dsd3/tb_serial_comparator_auto.v(95)
#    Time: 406 ns   Iteration: 0   Instance: /tb_serial_comparator_auto
# Break in Module tb_serial comparator auto at C:/Users/Lenovo/Desktop/dsd3/tb seri
```

برای هر دو نوع مقایسه کننده نیز برای هر سه حالت (دو عدد مساوی، بزرگتر، و کوچکتر) هر کدام یک تست در کولتوس انجام دادیم که عکس خروجی آنها نیز در زیر آمده است:

مقایسه کننده اول:





برای مقایسه کننده دوم:

توی تست مقایسه کننده دوم 50 نانو ثانیه نشان دهنده زمان صفر شدن reset یا شروع ورودی ها است و بعد از ان در تصویر لحظه مشخص شدن وضعیت آن دو نسبت به مشخص است که تا انتهای کار حفظ شده است (البته چون equal از مرحله قبل 1 است برای حالت مساوی تغییری نمیکند این سیگنال و همان یک می ماند اما برای حالت بزرگتر یا کوچکتر واضحاً در تصویر لحظه یک شدن سیگنال مربوطه این سیگنال صفر می شود)

